

Patrones de diseño en Poo

MEMENTO FLYWEIGHT

**PRESENTADO POR TOMAS
MURILLO GIRALDO**



INTRODUCCIÓN

"En esta presentación vamos a hablar sobre dos patrones de diseño utilizados en programación orientada a objetos: Memento y Flyweight.

Aunque ambos son patrones de diseño, solucionan problemas diferentes.

Memento se enfoca principalmente en guardar y restaurar el estado de un objeto, mientras que Flyweight busca reducir el consumo de memoria compartiendo información que puede ser reutilizada entre varios objetos."



MEMENTO: ¿QUÉ PROBLEMA RESUELVE?

Permite guardar y restaurar el estado anterior de un objeto sin revelar sus detalles internos.

Problema

Un objeto cambia constantemente:

Estado A → Estado B → Estado C

¿Y si necesitamos volver al estado anterior?

Ejemplos

-  Ctrl + Z
-  Partidas guardadas
-  Editores de texto
-  Restaurar configuraciones



¿CÓMO FUNCIONA MEMENTO?

"Memento normalmente tiene tres participantes principales.

El primero es el Originator. Es el objeto cuyo estado queremos guardar.

Después tenemos el Memento, que es el objeto encargado de almacenar ese estado.

Y finalmente tenemos el Caretaker, que administra los diferentes estados guardados, pero no necesita conocer los detalles internos del objeto.

El proceso es sencillo: el Originator crea un Memento con su estado actual, el Caretaker lo guarda y, cuando necesitamos volver atrás, el Caretaker entrega ese Memento al Originator para que pueda restaurarlo."

01
originator

02
Memento

03
Caretaker

EJEMPLO

```
public string Texto { get; set; }

public EditorMemento Guardar()
{
    return new EditorMemento(Texto);
}

public void Restaurar(EditorMemento memento)
{
    Texto = memento.Texto;
}
}

class EditorMemento
{
    public string Texto { get; }

    public EditorMemento(string texto)
    {
        Texto = texto;
    }
}
```

VENTAJAS Y DESVENTAJAS DE MEMENTO

Ventajas

- *Permite implementar deshacer y rehacer.*
- *Protege el estado interno del objeto.*
- *Facilita recuperar estados anteriores.*
- *Separa la gestión del historial del objeto principal.*

Deventajas

- *Puede consumir bastante memoria.*
- *Guardar muchos estados puede ser costoso.*
- *Aumenta la cantidad de clases/objetos.*

FLYWEIGHT: ¿QUÉ PROBLEMA RESUELVE?

Flyweight

Reduce el consumo de memoria compartiendo información entre objetos similares.

Problema

Imaginemos un videojuego:



1000 árboles → ¿necesitamos guardar toda la información 1000 veces?

¿ COMO FUNCIONA FLYWEIGHT?

"Ahora pasamos al patrón Flyweight.

Flyweight tiene un objetivo completamente diferente a Memento. Su objetivo principal es reducir el consumo de memoria.

Pensemos en un videojuego que tiene miles de árboles. Cada árbol podría tener información como textura, modelo, color y otras características que son iguales para muchos árboles.

En lugar de guardar toda esa información repetida en cada objeto, Flyweight permite compartir la información que es común y mantener solamente los datos que son diferentes para cada árbol."

ESTADO = INTRÍNSECO Y EXTRÍNSECO

01

Se puede compartir



Modelo

Textura

Tamaño base

02

Es específico de
cada objeto



Posición

Rotación

Escala

EJEMPLO

```
class Arbol
{
    public string Tipo { get; }

    public Arbol(string tipo)
    {
        Tipo = tipo;
    }
}

class ArbolFactory
{
    private Dictionary<string, Arbol> arboles = new();

    public Arbol ObtenerArbol(string tipo)
    {
        if (!arboles.ContainsKey(tipo))
            arboles[tipo] = new Arbol(tipo);

        return arboles[tipo];
    }
}
```

MEMENTO VS FLYWEIGHT

	MEMENTO	FLYWEIGHT
Tipo	Comportamiento	Estructural
Objetivo	Guardar estados	Ahorrar memoria
Idea principal	Restaurar	Compartir
Ejemplo	Ctrl + Z	Video juegos
Elemento clave	Estado anterior	Objetos reutilizables

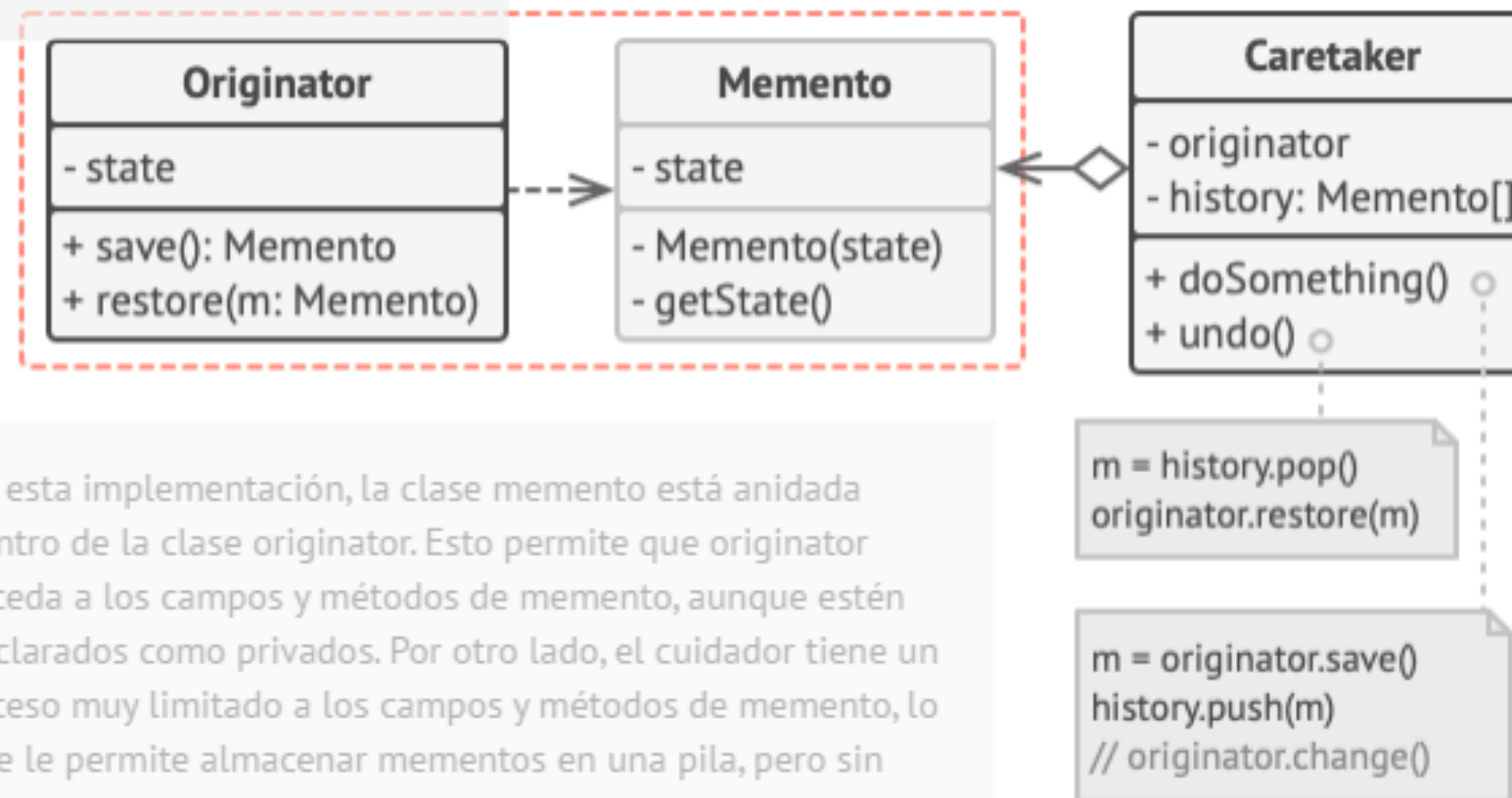
CONCLUSIONES

"En conclusión, los patrones de diseño nos permiten utilizar soluciones ya conocidas para problemas comunes de programación. Memento nos ayuda a manejar estados anteriores de los objetos, mientras que Flyweight nos ayuda a optimizar recursos evitando almacenar información repetida. Ambos patrones muestran cómo los principios de la programación orientada a objetos pueden utilizarse para construir sistemas más organizados y eficientes."

Diagrama de clase Memento

1 La clase **Originator** puede generar instantáneas de su propio estado, así como restaurar su estado a partir de dichas instantáneas cuando sea necesario.

2 El **objeto Memento** es un objeto de valor que actúa como una instantánea del estado del originador. Es práctica común hacer que el objeto Memento sea inmutable y pasarle los datos solo una vez, a través del constructor.



4 En esta implementación, la clase memento está anidada dentro de la clase originator. Esto permite que originator acceda a los campos y métodos de memento, aunque estén declarados como privados. Por otro lado, el cuidador tiene un acceso muy limitado a los campos y métodos de memento, lo que le permite almacenar mementos en una pila, pero sin modificar su estado.

3 El **Guardián** sabe no solo "cuándo" y "por qué" capturar el estado del originador, sino también cuándo se debe restaurar dicho estado.

Un custodio puede llevar un registro del historial del creador almacenando una pila de recuerdos. Cuando el creador necesita viajar al pasado, el custodio toma el recuerdo que se encuentra en la parte superior de la pila y se lo entrega al método de restauración del creador.

Diagrama de clase Flyweigth

